COMS4040A Assignment 1 – Report

Liam Binedell, 2574850, Coms Hons

April 3, 2026

1 Problem 1: Image Convolution Using CUDA C/C++

1.1 Description of Implementation and Experiments

1.1.1 Overview

Image convolution is a fundamental operation in image processing in which each output pixel is computed as a weighted sum of its neighbouring input pixels. Given an image $Y \in \mathbb{Z}^{M \times N}$ and a convolution kernel $F \in \mathbb{Z}^{L \times P}$, the discrete 2D convolution is defined as:

$$(Y * F)[i, j] = \sum_{l=-a}^{a} \sum_{p=-b}^{b} Y[i+l, j+p] F[l+a, p+b], \quad 0 \leq i < M, 0 \leq j < N \tag{1}$$

where $a = \lfloor L/2 \rfloor$ and $b = \lfloor P/2 \rfloor$. Pixels outside the image boundary are treated as zero. Three filters were implemented and evaluated: a 3×3 sharpening filter, a 5×5 emboss filter, and an averaging filter tested across kernel sizes $k \in \{3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25\}$. All experiments were performed on a 512×512 greyscale PGM image.

Each filter was implemented using three methods:

1. **Serial** – a sequential CPU implementation used as a baseline reference.
2. **CUDA Global Memory** – a parallel GPU implementation where each thread reads directly from global device memory.
3. **CUDA Shared Memory** – a parallel GPU implementation using cooperative tiled loading into shared memory to reduce redundant global memory accesses.

1.1.2 Image Loading and Storage

PGM images were loaded and written using custom C functions, as no suitable CUDA SDK utility was identified for this purpose. The image data is stored as a flat 1D array of `unsigned char`, where pixel (x, y) maps to flat index

$y \times \text{width} + x$. This layout is preferred for GPU workloads as it maps directly onto linear device memory and facilitates coalesced memory access patterns, whereby adjacent threads in a warp access consecutive memory addresses within the same row.

1.1.3 Serial Implementation

The serial implementation iterates over every pixel using nested row and column loops. For each pixel, a second pair of nested loops iterates over the filter window centred on that pixel. Out-of-bounds neighbours are treated as contributing zero to the weighted sum. The result is clamped to $[0, 255]$ before being written to a separate output buffer to avoid in-place corruption of the input data, which would cause later pixels to be computed from already-modified values.

1.1.4 CUDA Global Memory Implementation

The global memory kernel assigns one thread per output pixel using a 2D thread and block layout. A 16×16 thread block (256 threads) was chosen, with grid dimensions computed dynamically as:

$$\text{grid} = \left(\left\lceil \frac{W}{16} \right\rceil, \left\lceil \frac{H}{16} \right\rceil \right) \tag{2}$$

Each thread independently computes its pixel coordinate from `blockIdx` and `threadIdx`, applies the convolution by reading neighbour pixels from global device memory, and writes the clamped result to a separate output buffer. A boundary guard at the top of the kernel causes threads whose coordinates fall outside the image dimensions to exit immediately without performing any memory access.

The sharpen and emboss filter weights are stored in CUDA `__constant__` memory rather than passed as a pointer to global memory. Since all threads read the same filter values, constant memory's broadcast caching mechanism eliminates redundant global memory transactions across threads in the same warp, improving effective memory throughput.

1.1.5 CUDA Shared Memory Implementation

The shared memory kernel improves upon the global memory version by exploiting data reuse between threads within a block. For a $k \times k$ filter, each input pixel may be needed by up to k^2 threads. Rather than each thread independently fetching its neighbourhood from global memory, all threads in a block cooperatively load a tile of the input image into shared memory once, then read their filter windows from the tile.

The shared memory tile is padded by a halo of width $\lfloor k/2 \rfloor$ on each side to accommodate the neighbourhood pixels required by threads at the block boundary. The tile dimensions are:

$$\text{tile_width} = \text{block_width} + 2 \times \lfloor k/2 \rfloor \tag{3}$$

A strided cooperative loading strategy was adopted, where all threads collectively iterate over all tile positions in steps of the block size. Each position maps back to its corresponding global image coordinate, with out-of-bounds positions set to zero. This approach guarantees complete and correct tile coverage for any filter size, without the complexity and

correctness risks of per-edge conditional loading. A `__syncthreads()` barrier follows the loading phase to ensure all tile data is committed to shared memory before any thread begins reading from it for computation.

The averaging filter does not use `__constant__` memory for weights since all weights are implicitly uniform – each in-bounds pixel within the window contributes equally, and the accumulated sum is divided by k^2.

1.1.6 Timing Methodology

Execution times were measured using `gettimeofday()` on the host. The timing interval captures only the kernel execution time, excluding host-to-device and device-to-host memory transfers. For CUDA kernels, `cudaDeviceSynchronize()` was called between the kernel launch and the end timestamp to block the CPU until the GPU completes, since CUDA kernel launches are asynchronous by default. The elapsed time is computed as:

$$t = (t_{\text{end}}.\texttt{tv_sec} - t_{\text{start}}.\texttt{tv_sec}) + \frac{t_{\text{end}}.\texttt{tv_usec} - t_{\text{start}}.\texttt{tv_usec}}{10^6} \tag{4}$$

1.2 Performance Results

1.2.1 Sharpening and Embossing

Table 1 presents the execution times for sharpening and embossing across all three implementations. Speedup is computed relative to the serial baseline.

Table 1: Execution times and speedup for sharpening and embossing filters.

Filter	Implementation	Time (s)	Speedup
Sharpen (3×3)	Serial	0.002854	1.00×
	CUDA Global	0.000209	13.66×
	CUDA Shared	0.000060	47.57×
Emboss (5×5)	Serial	0.006875	1.00×
	CUDA Global	0.001197	5.74×
	CUDA Shared	0.000204	33.70×

1.2.2 Averaging Filter

Table 2 presents execution times across all three implementations for averaging kernel sizes from 3×3 to 25×25.

Table 2: Execution times (seconds) for averaging filter across kernel sizes.

Kernel	Serial (s)	Global (s)	Shared (s)	Speedup (Global)	Speedup (Shared)
3×3	0.001615	0.000070	0.000058	23.07×	27.84×
5×5	0.003723	0.000063	0.000060	59.10×	62.05×
7×7	0.008824	0.000113	0.000066	78.09×	133.70×
9×9	0.012554	0.000124	0.000073	101.24×	171.97×
11×11	0.018816	0.000134	0.000115	140.42×	163.62×
13×13	0.024255	0.000138	0.000117	175.76×	207.31×
15×15	0.036064	0.000142	0.000099	253.97×	364.28×
17×17	0.044178	0.000173	0.000116	255.36×	380.84×
19×19	0.055494	0.000179	0.000134	309.91×	414.13×
21×21	0.064123	0.000195	0.000156	328.84×	411.05×
23×23	0.076231	0.000206	0.000160	369.96×	476.44×
25×25	0.088804	0.000231	0.000177	384.43×	501.72×

1.3 Discussion and Analysis of Results

1.3.1 Serial vs CUDA Performance

Both CUDA implementations substantially outperform the serial baseline across all filters and kernel sizes, demonstrating the effectiveness of GPU parallelism for this workload. The serial implementation's execution time grows roughly proportional to k^2 as the filter size increases, since each pixel requires k^2 multiply-accumulate operations. For the 25×25 averaging filter, the serial implementation takes 0.0888 seconds, while the CUDA global and shared implementations take 0.000231 and 0.000177 seconds respectively – speedups of approximately 384× and 502×.

1.3.2 Global vs Shared Memory

The shared memory implementation consistently outperforms the global memory implementation, with the advantage growing as filter size increases. For the 3×3 sharpening filter the shared memory version is approximately 3.5× faster than global memory, while for the 25×25 averaging filter it is approximately 1.3× faster. The growing advantage at larger filter sizes is expected: each thread reads more neighbours from memory, increasing the degree of data sharing between threads and therefore the benefit of caching those values in shared memory rather than repeatedly fetching them from global memory.

The emboss filter shows the most pronounced global-to-shared speedup ratio (5.87×), which is partly attributable to its larger 5×5 filter window relative to sharpening, and possibly to the specific access pattern of its sparse weights interacting less favourably with the global memory cache hierarchy.

1.3.3 Scaling Behaviour

As shown in Table 2, the speedup of both CUDA implementations over serial increases consistently with kernel size. This is because the serial implementation's cost scales as $O(k^2)$ per pixel, whereas the GPU implementations amortise their launch overhead across $512 \times 512 = 262{,}144$ parallel threads, and the per-thread cost increase from larger kernels

is masked by the GPU's ability to hide memory latency through warp scheduling. The shared memory implementation maintains a consistent advantage over global memory at all sizes tested.

1.3.4 First-Run Warm-Up Effect

It was observed during testing that the first CUDA kernel launch in a session consistently produced a much higher execution time than subsequent launches – typically an order of magnitude higher. This is a well-known GPU warm-up phenomenon caused by one-time CUDA runtime initialisation, context creation, and JIT compilation occurring on the first launch. Reported timings were taken from stable subsequent runs and are not affected by this artefact.

1.4 Challenges Faced and Solutions

PGM file parsing. The test image contained a comment line in its header, which caused the width, height, and maximum value fields to be misread when the comment was not properly skipped. This led to incorrect `malloc` sizes and intermittent segmentation faults. The fix was to use `fscanf(f, " %c", &c)` with a leading space to skip whitespace before reading the first character, ensuring the parser correctly identified and consumed comment lines.

Timing instability. Early timing results were negative or wildly variable because only `tv_usec` was subtracted, which overflows when the operation crosses a one-second boundary. This was corrected by incorporating `tv_sec` into the elapsed time calculation.

Asynchronous kernel execution. Initial CUDA timing results were incorrectly fast because `gettimeofday()` was called immediately after the kernel launch before the GPU had completed. Adding `cudaDeviceSynchronize()` between the launch and the end timestamp resolved this.

Shared memory halo correctness. An initial per-edge conditional halo loading strategy for the variable-size averaging kernel produced incorrect output for larger filter sizes due to missing corner pixels. This was replaced with a strided cooperative loading loop that iterates all threads over all tile positions, guaranteeing complete coverage for any kernel size.

In-place convolution. An early implementation wrote convolution results back into the input buffer while still reading from it. This caused later pixels to be computed from already-modified values, producing incorrect output. This was resolved by allocating a separate output buffer and writing results there exclusively.

1.5 Conclusion

This section presented three implementations of 2D image convolution applied to sharpening, embossing, and averaging filters on a 512×512 greyscale image. The CUDA implementations achieved substantial speedups over the serial baseline, with the shared memory implementation consistently outperforming the global memory version. Speedups increased with filter size, reflecting the GPU's ability to exploit parallelism and data reuse more effectively as per-pixel workload grows. The results demonstrate that shared memory tiling is an effective and important optimisation for convolution workloads on GPU hardware, particularly for larger filter kernels where data sharing between threads is most significant.